

Sure bro. Below is a clean **EOD document** for today's Jenkins Scripted CI/CD task, including the issue you faced and its solution.

EOD – Jenkins Scripted CI/CD Pipeline

Date: 06 October 2026

Task: Setup Jenkins CI/CD Pipeline using Scripted Pipeline

Application: Tomcat Web Application

Branch: main

Repository: https://github.com/Zeeshancloud15/Tomcat_app01.git

1. Objective

The objective was to configure a Jenkins **Scripted Pipeline** for a Java web application with the following CI/CD stages:

```
graph TD; A[Git Checkout] --> B[Maven Build]; B --> C[WAR Validation]; C --> D[SonarQube Analysis]; D --> E[SonarQube Quality Gate]; E --> F[Upload WAR to Nexus]; F --> G[Deploy to Tomcat]; G --> H[Slack Notification];
```

Git Checkout
↓
Maven Build
↓
WAR Validation
↓
SonarQube Analysis
↓
SonarQube Quality Gate
↓
Upload WAR to Nexus
↓
Deploy to Tomcat
↓
Slack Notification

2. Jenkins Job Creation

Created a new Jenkins Pipeline job:

Job Name:

jenkins-scripted

Pipeline configuration:

Definition: Pipeline script from SCM

SCM: Git

Repository:

https://github.com/Zeeshancloud15/Tomcat_app01.git

Script Path:

Jenkinsfile

The Jenkinsfile is maintained in Git rather than being directly written inside the Jenkins job.

3. Pipeline Type

The task required a **Scripted Pipeline**.

Therefore, the Jenkinsfile uses:

```
node {  
  
    stage('Stage Name') {  
        // commands  
    }  
  
}
```

It does **not** use the Declarative Pipeline structure:

```
pipeline {  
    agent any  
  
    stages {  
        ...  
    }  
}
```

4. Maven Configuration

Jenkins Maven installation:

```
maven-integration
```

The pipeline loads the configured Maven installation:

```
def mavenTool = tool 'maven-integration'  
env.PATH = "${mavenTool}/bin:${env.PATH}"
```

This allows the pipeline to execute Maven commands such as:

```
mvn clean package
```

5. Pipeline Stages

Stage 1 – Source Checkout

The application source code is checked out from GitHub.

```
stage('Source Checkout') {  
    git(  
        branch: 'main',  
        url: 'https://github.com/ZeeShanCloud15/Tomcat\_app01.git'  
    )  
}
```

Stage 2 – Compile and Package

Maven is used to clean the previous build and create the WAR file.

```
stage('Compile and Package') {  
    sh 'mvn clean package'  
}
```

Expected artifact:

```
target/tomcat-app01.war
```

Stage 3 – Artifact Validation

The pipeline verifies that the WAR file was successfully generated.

```
stage('Artifact Validation') {  
    sh '''  
        ls -lh target/  
        test -f target/tomcat-app01.war  
    '''  
}
```

If the WAR file does not exist, this stage fails.

Stage 4 – SonarQube Analysis

SonarQube analysis is performed using the configured Jenkins SonarQube server:

sonar-server

Pipeline:

```
stage('Code Quality Analysis') {  
    withSonarQubeEnv('sonar-server') {  
        sh '''  
            /opt/sonar-scanner/bin/sonar-scanner  
        '''  
    }  
}
```

SonarQube checks the application for issues such as:

- Bugs
- Vulnerabilities
- Code smells
- Code duplication
- Maintainability issues
- Reliability issues

Stage 5 – Quality Gate

After SonarQube analysis, Jenkins waits for the Quality Gate result.

```

stage('Quality Validation') {
    timeout(time: 1, unit: 'MINUTES') {
        waitForQualityGate abortPipeline: false
    }
}

```

The Quality Gate determines whether the code meets the configured quality requirements.

Stage 6 – Publish Artifact to Nexus

The generated WAR file is uploaded to Nexus Repository.

```

stage('Publish Artifact') {
    nexusArtifactUploader(
        nexusVersion: 'nexus3',
        protocol: 'http',
        nexusUrl: '3.95.22.37:8081',
        groupId: 'com.customer',
        version: '2.0',
        repository: 'maven-release',
        credentialsId: 'nexus-creid',
        artifacts: [
            [
                artifactId: 'customer-web',
                classifier: '',
                file: 'target/tomcat-app01.war',
                type: 'war'
            ]
        ]
    )
}

```

Nexus is used as the artifact repository so that the generated WAR can be stored and reused.

Stage 7 – Deploy to Tomcat

The WAR file is deployed to the Tomcat server.

Tomcat server:

<http://35.169.107.79:8080>

Deployment context:

/customerapp

The deployment uses the Jenkins credential:

tomcat-jenkins

Stage 8 – Slack Notification

After successful deployment, Jenkins sends a Slack notification.

```
slackSend(  
    color: 'good',  
    message: ""  
CI/CD PIPELINE SUCCESS
```

```
Application : ${APPLICATION_NAME}  
Job          : ${env.JOB_NAME}  
Build       : #${env.BUILD_NUMBER}  
Status      : SUCCESS  
""  
)
```

If the pipeline fails, a failure notification is sent using:

```
color: 'danger'
```

6. Error Faced During Execution

Error

The first pipeline execution failed with:

```
org.codehaus.groovy.control.MultipleCompilationErrorsException:  
startup failed:
```

```
WorkflowScript: 1: unexpected char: '`'  
@ line 1, column 1.
```

```
```groovy
^
```

## Root Cause

The Jenkinsfile contained Markdown code-block characters:

```
```groovy
```

at the beginning of the file.

These characters are used by ChatGPT/Markdown to display code, but they are **not valid Groovy syntax**.

Jenkins therefore tried to interpret the backtick character ` as Groovy code and failed during compilation.

Solution

Removed the Markdown characters:

```
```groovy
```

and:

from the Jenkinsfile.

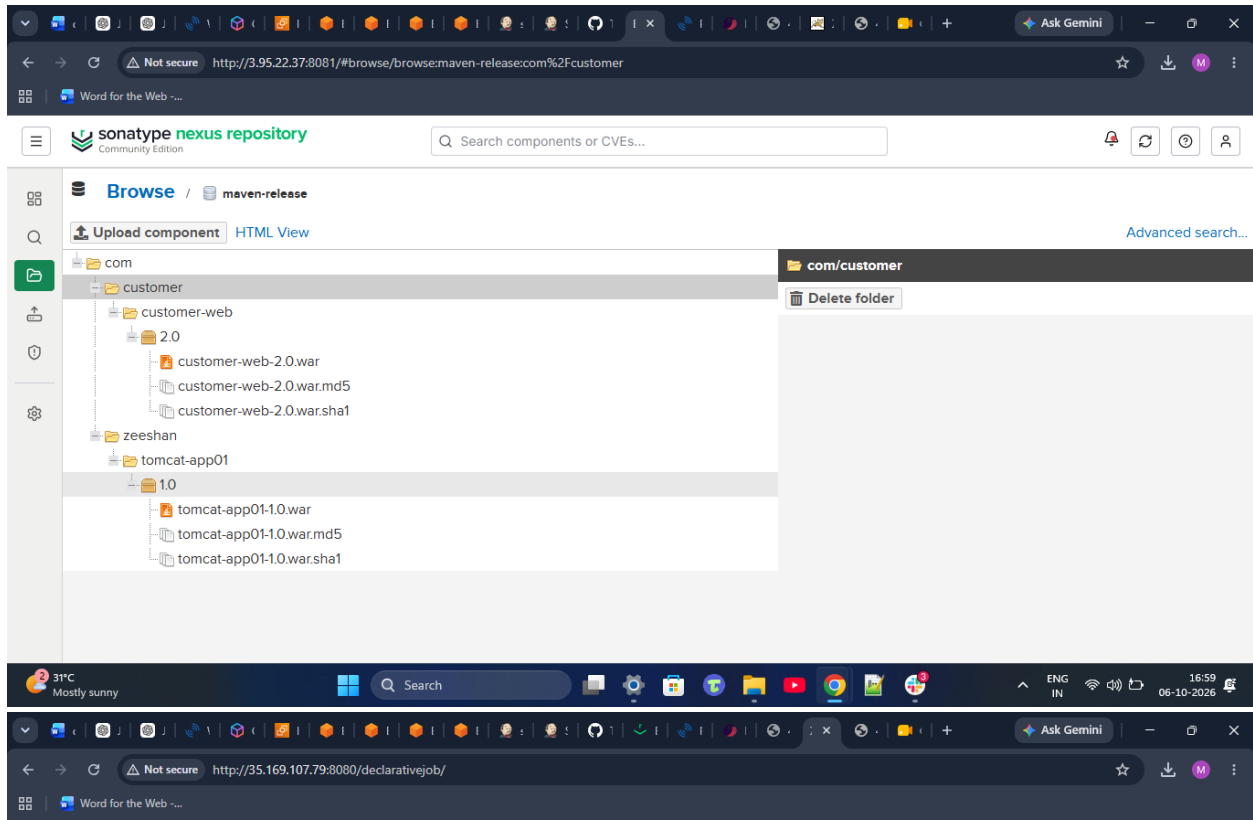
The file must start directly with:

```
```groovy
node {
```

and end with:

```
}
```

After committing the corrected Jenkinsfile to Git, Jenkins was able to read the pipeline script correctly.



Welcome to Zeeshan Cloud Tech

Jenkins CI/CD Deployment Successful!

Application: Tomcat App01

GitHub â†’ Jenkins â†’ SonarQube â†’ Maven â†’ Nexus â†’ Slack â†’ Tomcat



The top screenshot shows the Jenkins web interface for the 'scripted-pipeline-job'. The job status is 'Failure' (indicated by a red 'X'). The 'Stage View' table displays the following data:

	Source Checkout	Compile and Package	Artifact Validation	Code Quality Analysis	Quality Validation	Publish Artifact	Application Deployment	Notify Success
Average stage times: (full run time: ~15s)	199ms	3s	296ms	7s	94ms	101ms	77ms	103ms
#6 Oct 06 16:57 2 commits	199ms	3s	296ms	7s	94ms (paused for 2s)	101ms	77ms	103ms
#5 Oct 06 16:56 No Changes								

The bottom screenshot shows a Slack message in the '#jenkins-integration' channel. The message is from the 'Jenkins APP' and states 'CI/CD PIPELINE SUCCESS'. It includes the following details:

- Application : customer-web
- Job : scripted-pipeline-job
- Build : #6
- Status : SUCCESS
- Application URL: <http://35.169.107.79:8080/customerapp/>
- Jenkins Build: <http://44.195.88.168:8080/job/scripted-pipeline-job/6/>

RESULTS

The Jenkins Scripted Pipeline was configured with the complete CI/CD workflow:

- Git source checkout
- Maven compilation and packaging

- WAR validation
- SonarQube code analysis
- SonarQube Quality Gate
- Nexus artifact upload
- Tomcat deployment
- Slack success/failure notification
- Exception handling using try/catch
- Pipeline completion using finally

Key Learning

The major issue encountered was a **Groovy compilation error caused by Markdown backticks in the Jenkinsfile**. Removing the Markdown formatting allowed Jenkins to interpret the file as a valid Groovy Scripted Pipeline.